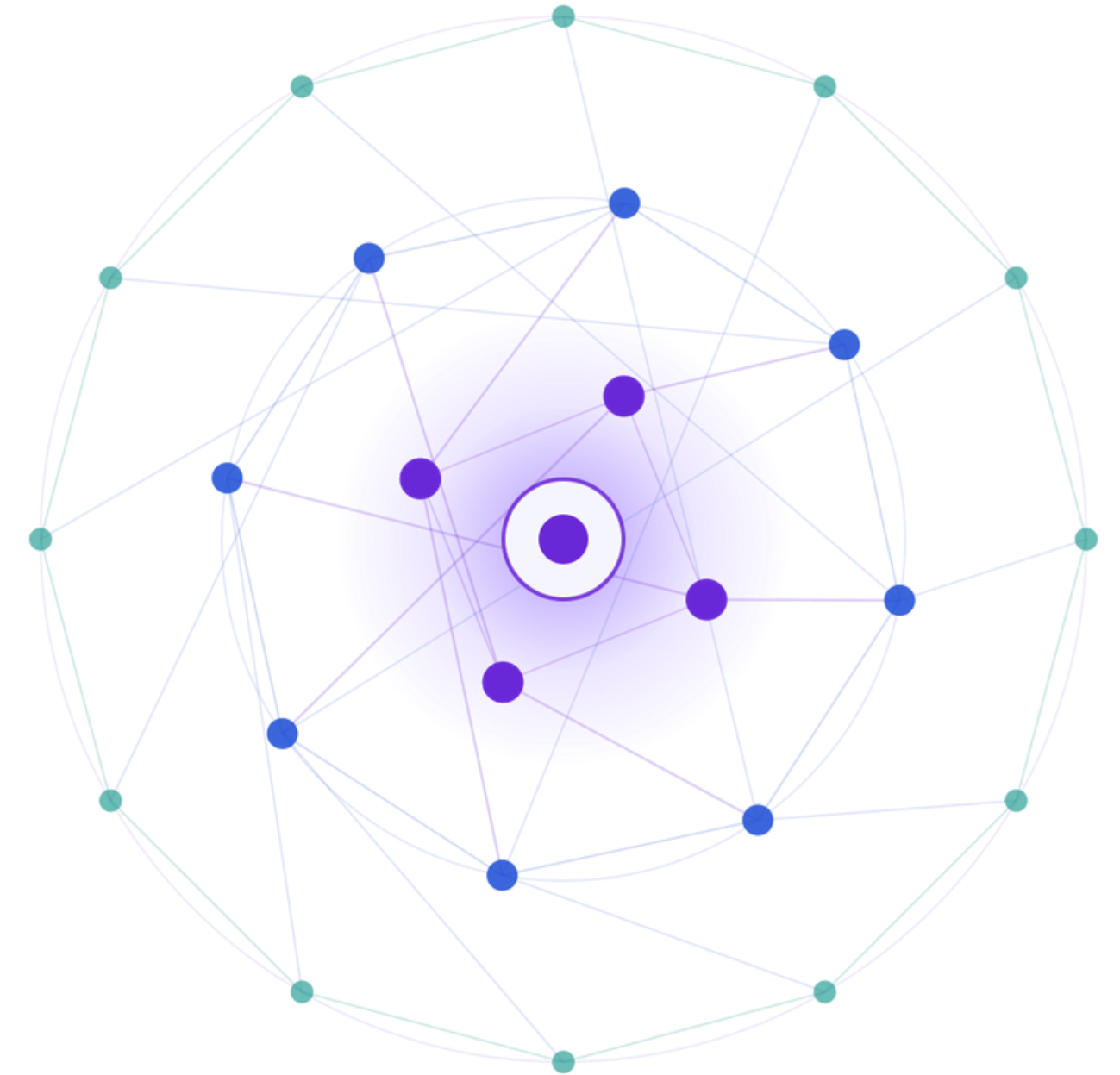


SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - THEME 4 Streaming Live RAG
- Team Name - SRMIST_VectorFlow
- College Name - S.R.M UNIVERSITY -K.T.R Campus
- Member Name & Email 1- PARTH TEWARI / pt1748@srmist.edu.in
- Member Name & Email 2- SHAURYA PRAKASH / Sp1426@srmist.edu.in
- Member Name & Email 3- ANUJ KUMAR SINGH / as9998@srmist.edu.in
- Member Name & Email 4- ANUBHAV BAL / ab8140@srmist.edu.in
- Submission Github link - <https://github.com/parhtework3056-commits/streaming-live-rag>



Theme - Streaming Live RAG

Real-Time Incremental Retrieval, Multi-Intent Decomposition & State-Preserving Answer Refinement

Problem Focus: Traditional RAG systems generally wait until the user finishes speaking before retrieval begins. This creates unnecessary latency and makes it difficult to handle evolving, multi-intent conversations.

Our Vision: Build an event-driven RAG system that can:

- Start retrieval while the user is still speaking.
- Decide intelligently between WAIT, RETRIEVE and SUPPRESS.
- Break compound questions into independent retrieval intents.
- Retrieve evidence from multiple sources in parallel.
- Preserve previous answer state.
- Update only the claims affected by newly arriving information.
- Provide citations and uncertainty indicators for factual claims.
- Expose a complete execution trace for every decision.

The system is designed as a corpus-grounded, closed-world RAG pipeline, where factual evidence must originate from the supplied corpus rather than external web search.

PROBLEM STATEMENT/STREAMING LIVE RAG

The Problem with Conventional RAG

Traditional RAG waits for the complete user query before retrieving information, causing high latency and poor real-time responsiveness. It struggles with multi-intent queries, noisy partial inputs, and late-arriving constraints, often requiring unnecessary reprocessing. It can also generate unsupported claims and lacks clear visibility into retrieval decisions and answer changes.

Our challenge:

How can we make RAG responsive enough to retrieve information while the user is still speaking, yet selective enough to avoid unnecessary retrieval — while preserving context, handling multiple intents, grounding every factual claim, and continuously refining the answer as new information arrives?

Our Goal

Existing Solutions & Gaps

Existing Approach: Conventional / Batch RAG

Complete Query → Retrieve → Generate → Answer

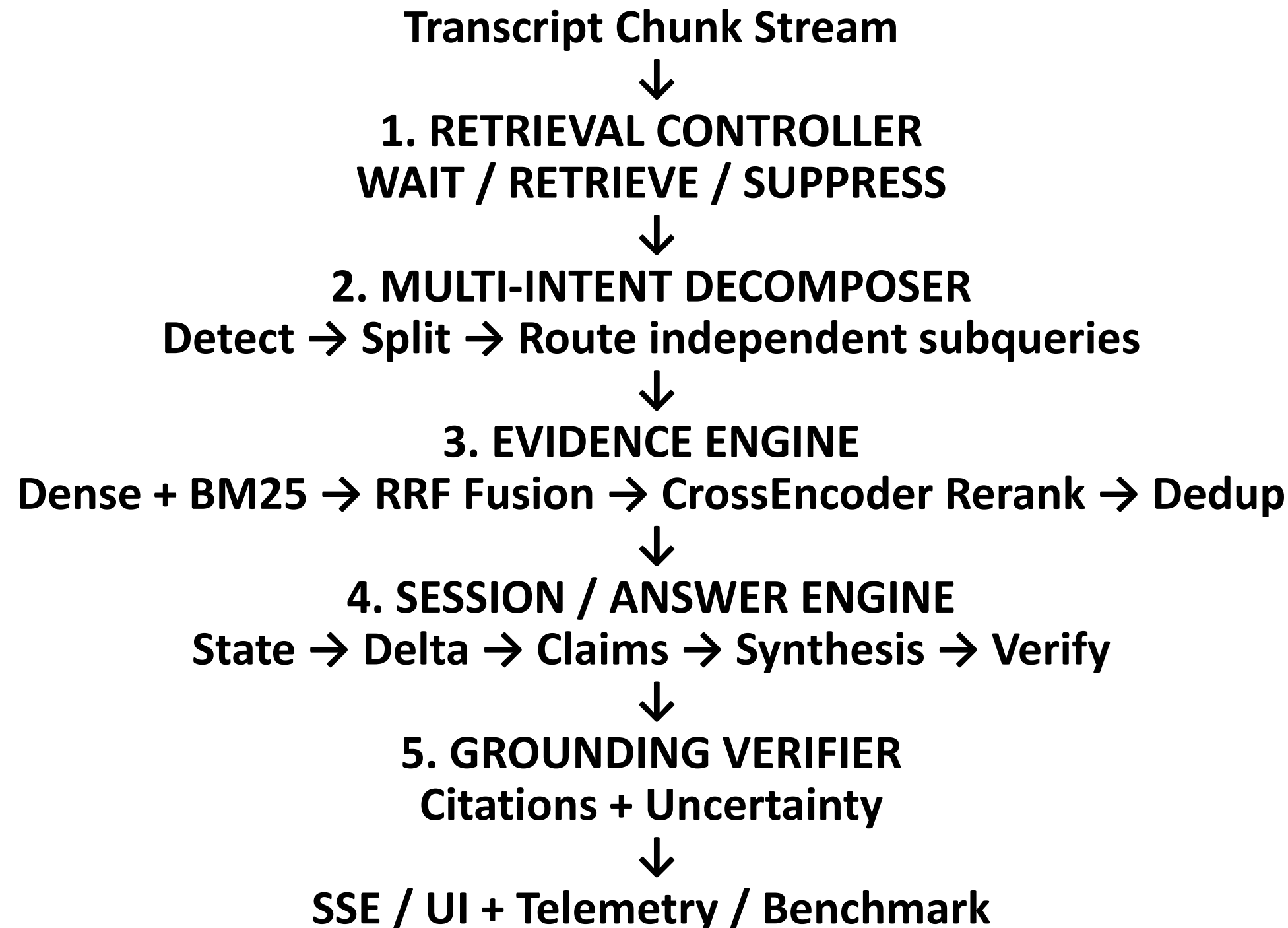
What it does well

- Simple and mature retrieval pipeline
- Effective for standalone queries
- Easy to implement and benchmark Critical Gaps
- Final-query retrieval → waits for utterance completion
- Single-query RAG → weak handling of compound intents
- Full regeneration → recomputes everything after late constraints
- Eager retrieval → may search from incomplete/noisy input
- Raw retrieved context → can contain redundant evidence
- Unverified generation → risk of unsupported claims
- Stateless responses → previous answer state may be lost
- Limited telemetry → difficult to explain system decisions Opportunity

Streaming + Adaptive Retrieval + Multi-Intent Understanding + Hybrid Retrieval + State Preservation + Claim-Level Grounding + Observability.

Our Solutions & Architecture Diagram

STREAMING LIVE RAG — EVENT-DRIVEN RETRIEVAL & ANSWER REFINEMENT



Key principle: every boundary is represented by a structured, testable event/object.

Demo & Product Walkthrough

HERO DEMO — INCREMENTAL MULTI-INTENT QUERY

0.0s: “I need to plan a customer workshop in...”

→ WAIT — intent incomplete.

0.8s: “...Pune for 30 people, and I need...”

→ RETRIEVE — provisional venue/capacity retrieval begins before the user finishes.

1.6s: “...the cancellation policy and catering options.”

→ DECOMPOSE — 3 sub-intents; parallel retrieval.

2.1s: Utterance ends

→ SYNTHESIZE — unified response with citations + uncertainty.

SECOND DEMO — LATE-ARRIVING DETAIL

Initial question → Answer V1 + citations

New constraint: “The trip was international and the booking was made after the travel.”

→ Detect semantic delta → identify affected claims → targeted retrieval → Answer V2

Unaffected claims/citations are preserved.

SUPPRESSION DEMO

“Repeat your last answer in three bullets.”

→ SUPPRESS; no new corpus retrieval; existing state is reformatted.

Tools and tech stack used

BACKEND / API

- Python
- FastAPI
- REST APIs
- Server-Sent Events (SSE)

RETRIEVAL

- Sentence Transformer embeddings
- Dense vector retrieval
- Qdrant / FAISS
- Sparse retrieval / BM25

RANKING

- Reciprocal Rank Fusion (RRF)
- CrossEncoder reranking
- Candidate fusion
- Deduplication

STATE & DATA

- Versioned JSON event contracts
- Stable chunk IDs
- Session state
- Evidence packs
- Claim records
- Answer versioning

FRONTEND & OBSERVABILITY

- React dashboard
- Real-time SSE visualization
- OpenTelemetry-compatible traces, metrics and logs

DEPLOYMENT

- Docker / Docker Compose
- Version-pinned dependencies
- Reproducible benchmark environment

Impact & Use case

IMPACT

- Reduced conversational latency by overlapping retrieval with user speech.
- Better handling of complex questions through parallel multi-intent retrieval.
- Efficient refinement by updating only affected claims.
- Improved factual reliability through claim-level grounding.
- Explainable AI through visible decisions, evidence and answer lineage.
- Corpus isolation keeps benchmark evidence restricted to approved sources.

USE CASES

- Customer Support — multiple questions and evolving details
- Travel & Policy Assistants — rules, conditions and booking constraints
- Enterprise Knowledge Assistants — internal document-grounded answers
- Meeting / Voice Assistants — retrieve while a user is speaking
- Healthcare / Compliance Knowledge Systems — evidence traceability and uncertainty
- Financial / Policy Assistants — precise policy-document retrieval

Target outcome: responsive answers without sacrificing evidence, continuity or auditability.

Innovation highlights, results and limitations

INNOVATION HIGHLIGHTS

1. Proactive retrieval while the user is speaking.
2. Adaptive WAIT / RETRIEVE / SUPPRESS controller.
3. Multi-intent parallel retrieval.
4. State-preserving answer refinement.
5. Hybrid dense + sparse retrieval with RRF and reranking.
6. Claim-level grounding and uncertainty handling.
7. Complete execution trace and judge-visible telemetry.
8. Closed-world corpus isolation.

LIMITATIONS

- Controller thresholds require benchmark tuning.
- Over-early retrieval can add noise/cost.
- Over-decomposition can increase latency.
- Poor chunking can reduce retrieval/citation quality.
- Insufficient evidence can still cause synthesis failure.
- Full speech recognition is outside the core benchmark scope.

OFFICIAL BENCHMARK TARGETS

- G1 Reproducibility — clean-machine automated replay
- G2 Early Retrieval — $\geq 80\%$ eligible queries
- G3 Multi-Intent — $\geq 70\%$ compound queries
- G4 Grounding — $\geq 85\%$ citation support
- G5 Session Refinement — verified continuity
- G6 Telemetry — 100% trace coverage

What's next

1. Benchmark Optimization

- Tune controller thresholds and semantic stability
- Improve intent decomposition
- Tune dense/sparse retrieval parameters

2. Advanced Retrieval

- Improve hybrid retrieval and reranking
- Better near-duplicate detection
- Adaptive evidence budgets

3. Real-Time Speech Integration

- Connect actual Speech-to-Text to the streaming transcript pipeline

4. Real-Time Speech Integration

- More precise semantic delta detection
- Claim dependency mapping
- Long-running conversational state
- Improved answer lineage

5. Production Deployment

- Scalable distributed architecture
- Secure enterprise corpus connectors
- Production-grade observability
- Fault-tolerant retrieval

6. Broader Applications

- Enterprise assistants
- Customer support
- Compliance and policy systems
- Knowledge management
- Real-time meeting assistants

Our Competitive Advantage

TRADITIONAL RAG

Query → Retrieve → Generate

OUR STREAMING LIVE RAG

Listen → Understand → Decide → Retrieve Early → Decompose → Parallel Retrieve → Refine → Verify → Explain

WHY WE STAND OUT

- ⚡ Proactive Retrieval — retrieval begins when enough semantic information is available.
- 🧠 Adaptive Decision Making — WAIT ≠ RETRIEVE ≠ SUPPRESS.
- 🔄 Multi-Intent Parallelism — independent evidence needs are retrieved concurrently.
- 🔄 Incremental Refinement — only affected claims are re-evaluated.
- 📌 Evidence-First Generation — synthesis receives a structured evidence pack.
- 🛡️ Fail-Closed Grounding — unsupported claims become uncertain rather than fabricated.
- 🔍 Judge-Visible Architecture — decisions, timing, evidence, versions and traces are visible.
- 🔒 Corpus Isolation — runtime factual evidence remains within the supplied corpus.

One-line pitch:

“Streaming Live RAG transforms query-then-retrieve RAG into an event-driven system that retrieves while the user speaks, handles multiple intents in parallel, preserves state, and continuously verifies factual claims against trusted evidence.”

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y/~~N~~)
- README with reproducible setup instructions (Y/~~N~~)
- Demo video, max 5 minutes (YouTube)(Y/~~N~~)
- Presentation file (PPT or PDF) (Y/~~N~~)

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India